Assignment 10

Chul Hee Kim

2021 11 23

Problem 1

a-) Out of 97 total observations, two predictors, *benign.prostatic.hyperplasia* and *capsular.penetration* have too many zeros. We need to first examine what these mean. Roughly 45% of 97 observations are zero for the two variables, so we may be tempted to completely ignore these two. But, a little bit of research showed that having 0 for those two metrics is common, especially for healthy men. Therefore, we will leave them as is in our dataset and start analyzing. The correlation matrix shows a few important variables, and we will use the best subset and stepwise both regression to find our model.

```
prostate <- getDataSet("Prostate Cancer.csv")
str(prostate)
```

```
## 'data.frame':    97 obs. of  8 variables:
##  $ PSA.level                   : num  0.651 0.852 0.852 0.852 1.448 ...
##  $ Cancer.volume               : num  0.56 0.372 0.601 0.301 2.117 ...
##  $ Weight                      : num  16 27.7 14.7 26.6 30.9 ...
##  $ Age                         : int  50 58 74 58 62 50 64 58 47 63 ...
##  $ Benign.prostatic.hyperplasia: num  0 0 0 0 0 ...
##  $ Seminal.vesicle.invasion    : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ Capsular.penetration        : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ Gleason.score               : int  6 7 7 6 6 6 6 6 7 6 ...
```

```
head(prostate)
```

```
##   PSA.level Cancer.volume Weight Age Benign.prostatic.hyperplasia
## 1     0.651        0.5599 15.959  50                            0
## 2     0.852        0.3716 27.660  58                            0
## 3     0.852        0.6005 14.732  74                            0
## 4     0.852        0.3012 26.576  58                            0
## 5     1.448        2.1170 30.877  62                            0
## 6     2.160        0.3499 25.280  50                            0
##   Seminal.vesicle.invasion Capsular.penetration Gleason.score
## 1                        0                   0            6
## 2                        0                   0            7
## 3                        0                   0            7
## 4                        0                   0            6
## 5                        0                   0            6
## 6                        0                   0            6
```

```
apply(prostate,2,function(x) sum(is.na(x))) # check any missing data
```

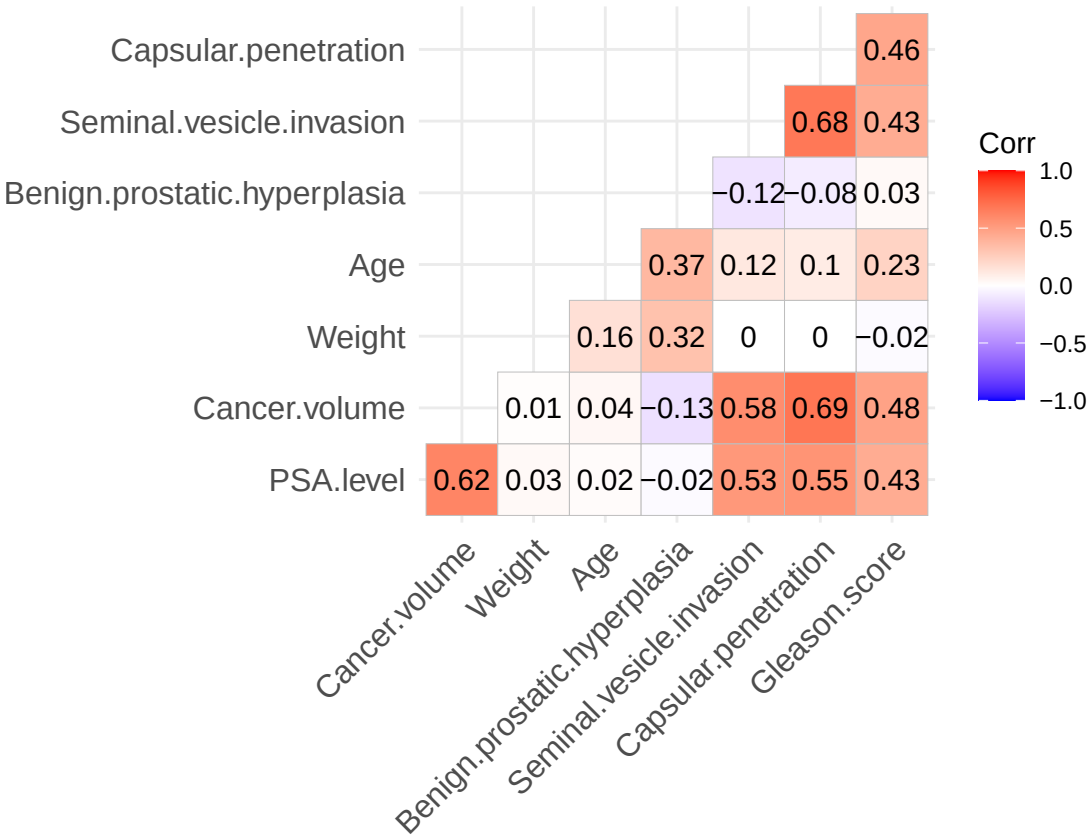
```
##                    PSA.level                Cancer.volume
##                            0                            0
##                       Weight                          Age
##                            0                            0
```

```
## Benign.prostatic.hyperplasia     Seminal.vesicle.invasion
##                            0                            0
##         Capsular.penetration                Gleason.score
##                            0                            0
```

```
apply(prostate,2,function(x) sum(x==0)) # check zeros
```

```
##                     PSA.level                Cancer.volume
##                             0                            0
##                        Weight                          Age
##                             0                            0
## Benign.prostatic.hyperplasia     Seminal.vesicle.invasion
##                            43                           76
##         Capsular.penetration                Gleason.score
##                            45                            0
```

```
corr <- round(cor(prostate),2)
ggcorrplot(corr, type="lower", lab=TRUE)
```




```
set.seed(567)
train.index <- sample(nrow(prostate), 65)
train.set <- prostate[train.index,]
test.set <- prostate[-train.index,]

f1 <- lm(PSA.level ~ ., data = train.set)
summary(f1)
```

```
##
## Call:
```

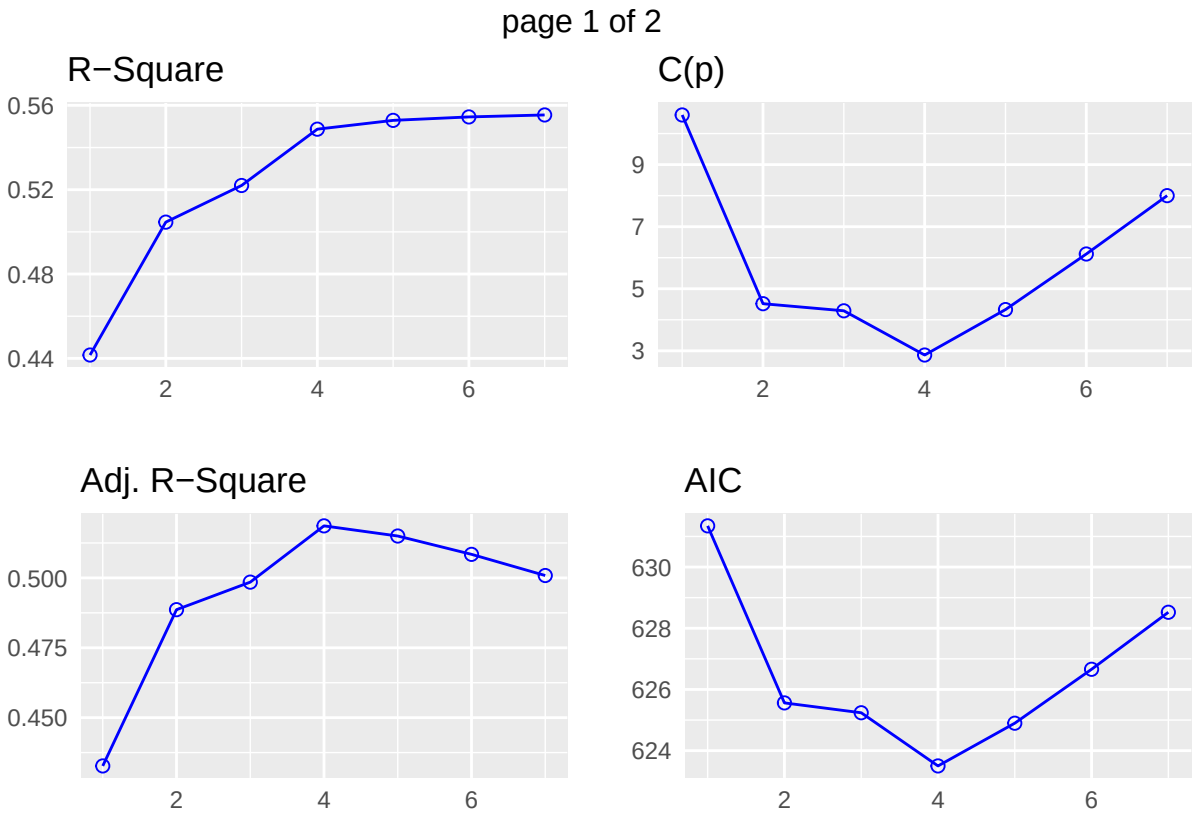
```
## lm(formula = PSA.level ~ ., data = train.set)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -58.959 -10.534   0.546   7.886 136.999
##
## Coefficients:
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                   36.0596    48.0216   0.751  0.45580
## Cancer.volume                  1.2769     0.6998   1.825  0.07327 .
## Weight                         0.4520     0.2845   1.589  0.11765
## Age                           -1.2624     0.5821  -2.169  0.03429 *
## Benign.prostatic.hyperplasia  -0.5805     1.6757  -0.346  0.73029
## Seminal.vesicle.invasion      -6.8214    13.9254  -0.490  0.62612
## Capsular.penetration           4.7785     1.5119   3.161  0.00252 **
## Gleason.score                  4.2182     5.7792   0.730  0.46844
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 28.31 on 57 degrees of freedom
## Multiple R-squared:  0.5555, Adjusted R-squared:  0.5009
## F-statistic: 10.17 on 7 and 57 DF,  p-value: 0.00000003443
```

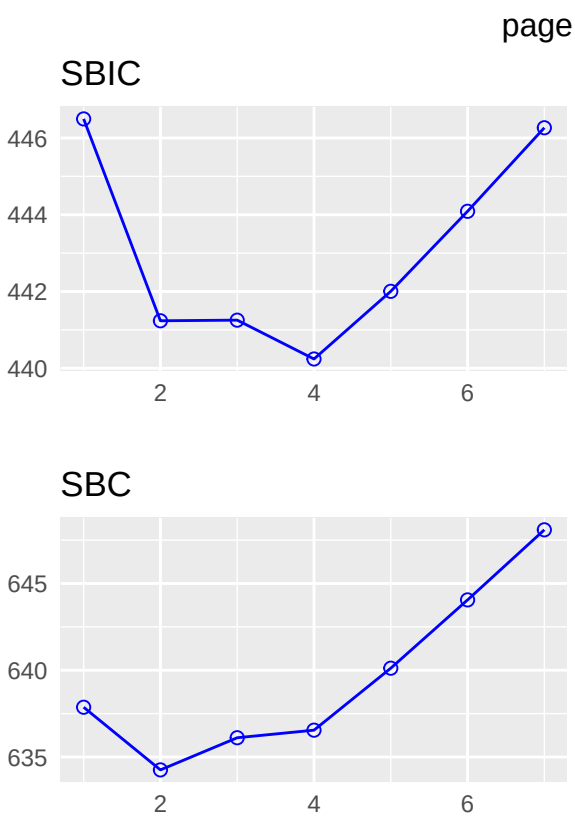
```
k1 <- ols_step_best_subset(f1)
k1
```

```
##                                                    Best Subsets Regression
## ---------------------------------------------------------------------------------------------------
## Model Index    Predictors
## ---------------------------------------------------------------------------------------------------
##      1         Capsular.penetration
##      2         Cancer.volume Capsular.penetration
##      3         Cancer.volume Age Capsular.penetration
##      4         Cancer.volume Weight Age Capsular.penetration
##      5         Cancer.volume Weight Age Capsular.penetration Gleason.score
##      6         Cancer.volume Weight Age Seminal.vesicle.invasion Capsular.penetration Gleason.score
##      7         Cancer.volume Weight Age Benign.prostatic.hyperplasia Seminal.vesicle.invasion Capsula
## ---------------------------------------------------------------------------------------------------
##
##                                                      Subsets Regression Summary
## ---------------------------------------------------------------------------------------------------
##                        Adj.        Pred
## Model    R-Square    R-Square    R-Square     C(p)        AIC         SBIC        SBC          MSEP
## ---------------------------------------------------------------------------------------------------
##   1        0.4416      0.4327      0.2351    10.6000    631.3454    446.4976    637.8685    59188.11
##   2        0.5046      0.4886      0.2953     4.5179    625.5601    441.2370    634.2577    53367.84
##   3        0.5220      0.4985      0.2891     4.2893    625.2385    441.2535    636.1104    52353.57
##   4        0.5487      0.5186      0.3087     2.8644    623.5008    440.2416    636.5471    50265.78
##   5        0.5529      0.5150      0.2979     4.3294    624.8970    442.0048    640.1178    50659.67
##   6        0.5545      0.5084      0.2452     6.1200    626.6593    444.0906    644.0544    51360.20
##   7        0.5555      0.5009      0.2382     8.0000    628.5225    446.2667    648.0920    52167.50
## ---------------------------------------------------------------------------------------------------
## AIC: Akaike Information Criteria
##  SBIC: Sawa's Bayesian Information Criteria
##  SBC: Schwarz Bayesian Criteria
```

```
##  MSEP: Estimated error of prediction, assuming multivariate normality
##  FPE: Final Prediction Error
##  HSP: Hocking's Sp
##  APC: Amemiya Prediction Criteria
```

```
plot(k1)
```






```
ols_step_both_p(f1, prem=0.05)
```

```
##
##                                    Stepwise Selection Summary
## -------------------------------------------------------------------------------------------
##                                  Added/                    Adj.
## Step          Variable          Removed     R-Square    R-Square      C(p)        AIC        RMSE
## -------------------------------------------------------------------------------------------
##    1    Capsular.penetration    addition       0.442       0.433    10.6000    631.3454    30.1757
##    2       Cancer.volume        addition       0.505       0.489     4.5180    625.5601    28.6499
## -------------------------------------------------------------------------------------------
```

The best subset regression shows that the choice could be Model 4 with *Cancer.volume, Weight, Age, Capsular.penetration*. But, using these four predictors results in a negative R^2 when assessed with the test set. The reason is that the OLS regression function has a negative beta for *Age*, so when the model is tested on the test set, many predicted PSA levels became negative and totally different from the observed values. We go back to the best subset regression summary and see that Model 4 has only a marginal advantage over Model 2 and 3. Our final choice is Model 2 because it is simpler and performs just as well. The stepwise both regression also confirms our choice.

```
m2 <- lm(PSA.level~Cancer.volume+Capsular.penetration, data=train.set)
summary(m2)
```

```
##
## Call:
## lm(formula = PSA.level ~ Cancer.volume + Capsular.penetration,
##     data = train.set)
##
## Residuals:
```

```
##     Min      1Q  Median      3Q     Max
## -67.191  -4.595   1.055   5.135 141.423
##
## Coefficients:
##                      Estimate Std. Error t value Pr(>|t|)
## (Intercept)            0.4517     4.7028   0.096  0.92379
## Cancer.volume          1.7197     0.6123   2.809  0.00664 **
## Capsular.penetration   3.7378     1.2663   2.952  0.00446 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 28.65 on 62 degrees of freedom
## Multiple R-squared:  0.5046, Adjusted R-squared:  0.4886
## F-statistic: 31.58 on 2 and 62 DF,  p-value: 0.0000000003493
```

```
dispRegFunc(m2)
```

```
## [1] "Y = 0.451691 + 1.7197Cancer.volume + 3.73781Capsular.penetration"
```

```
resampleMetrics(predict(m2, test.set), test.set$PSA.level)
```

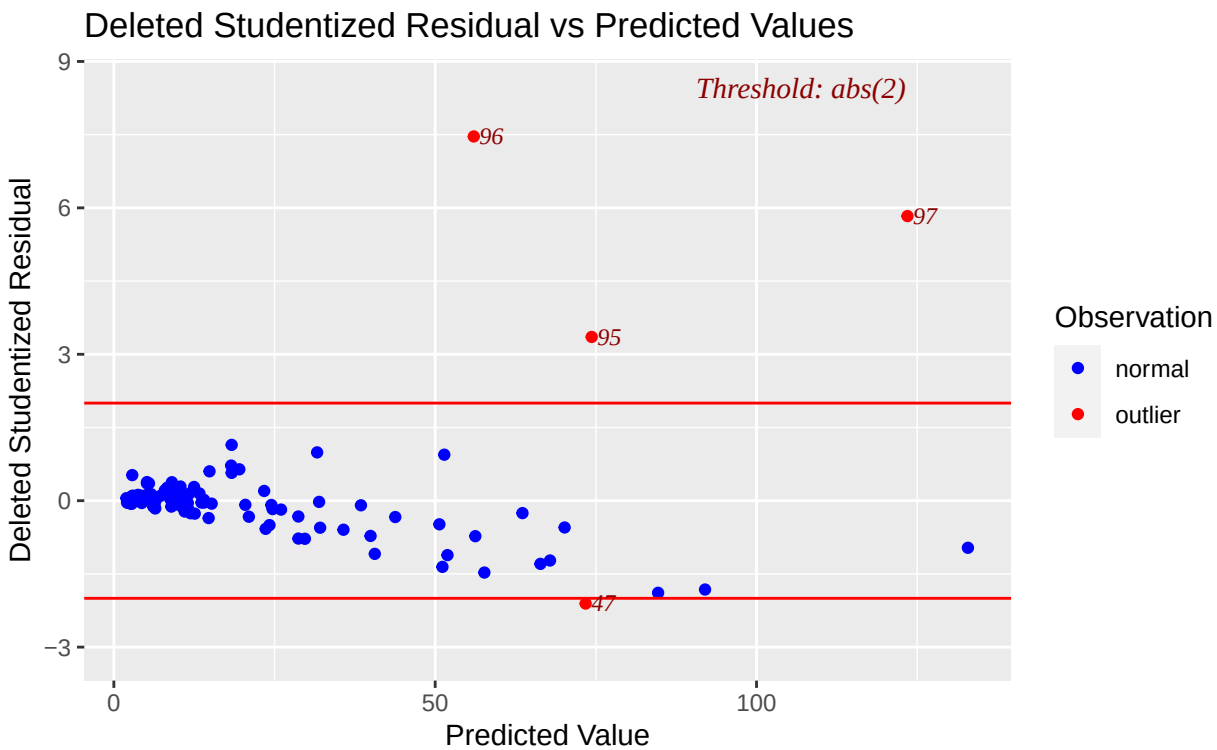
```
##             R2     RMSE      MAE
## [1,] 0.2116434 37.36241 17.61718
```

We should check the regression model assumptions and outliers before we talk about the model's performance and validity.

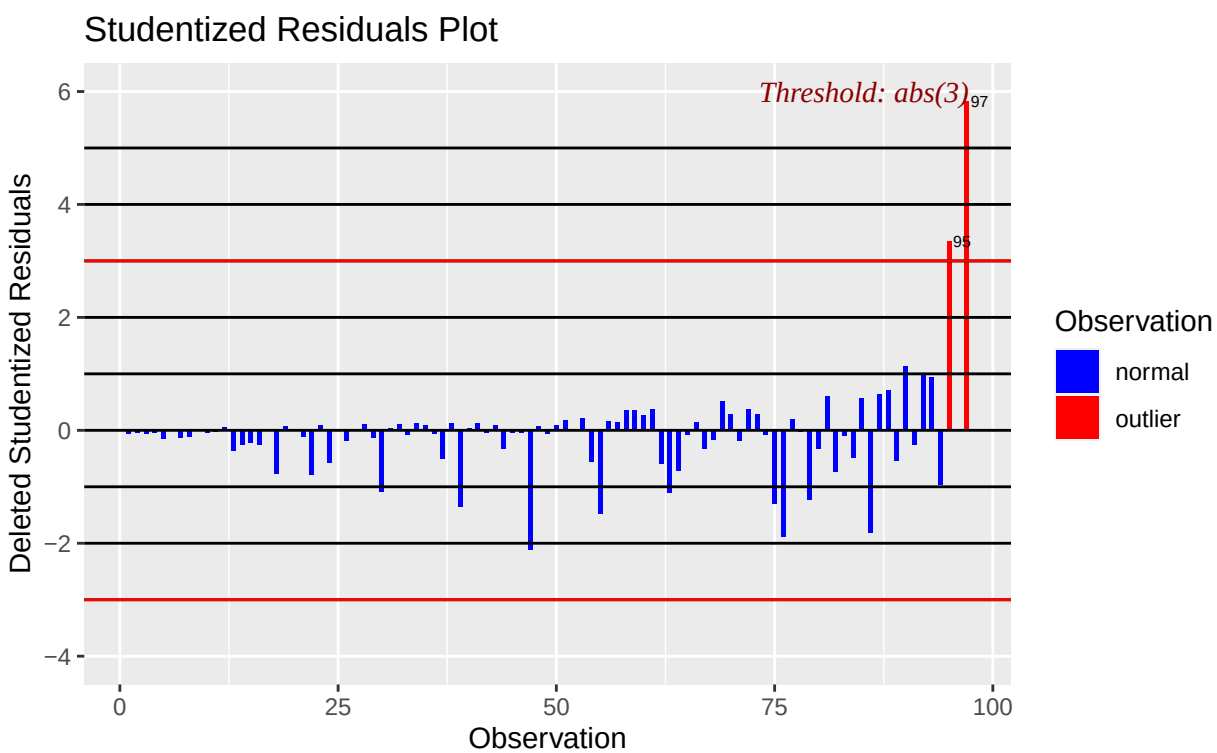
```
f1.1 <- lm(PSA.level ~ Cancer.volume+Capsular.penetration, data = prostate)
n <- nrow(prostate)
p <- length(f1.1$coefficients)

outlierTest(f1.1)
```

```
##    rstudent unadjusted p-value   Bonferroni p
## 96 7.463368  0.000000000044073 0.000000004275
## 97 5.831513  0.000000078974000 0.000007660400
```

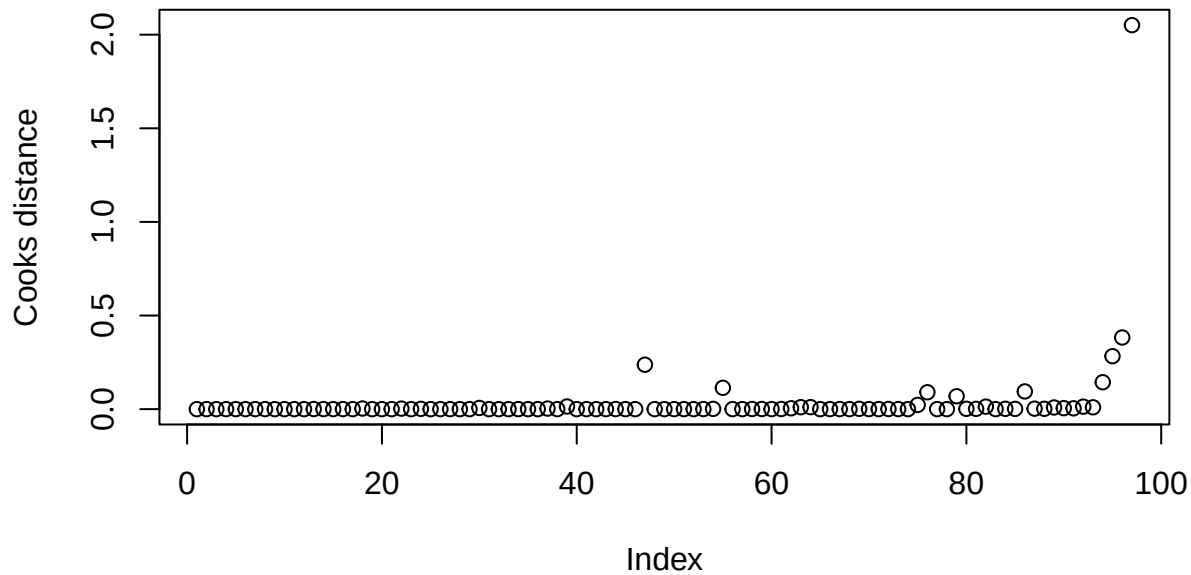
```
ols_plot_resid_stud_fit(f1.1)
```



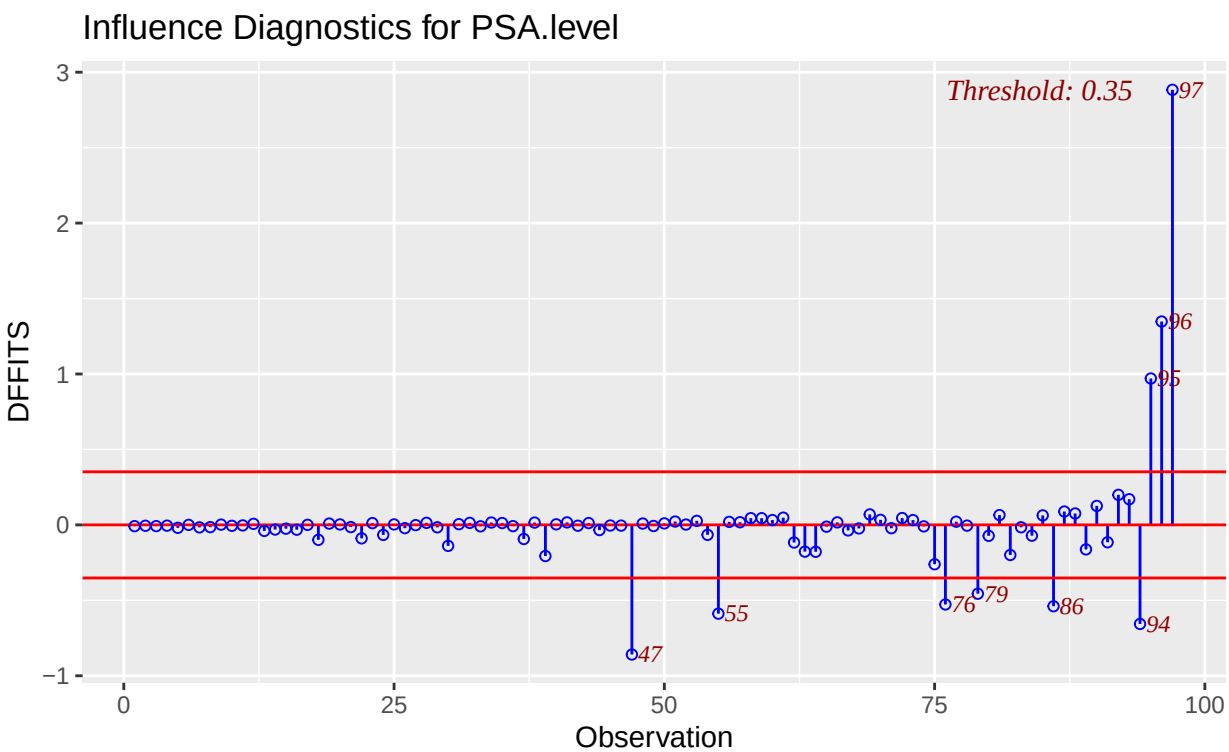

```
ols_plot_resid_stud(f1.1)
```



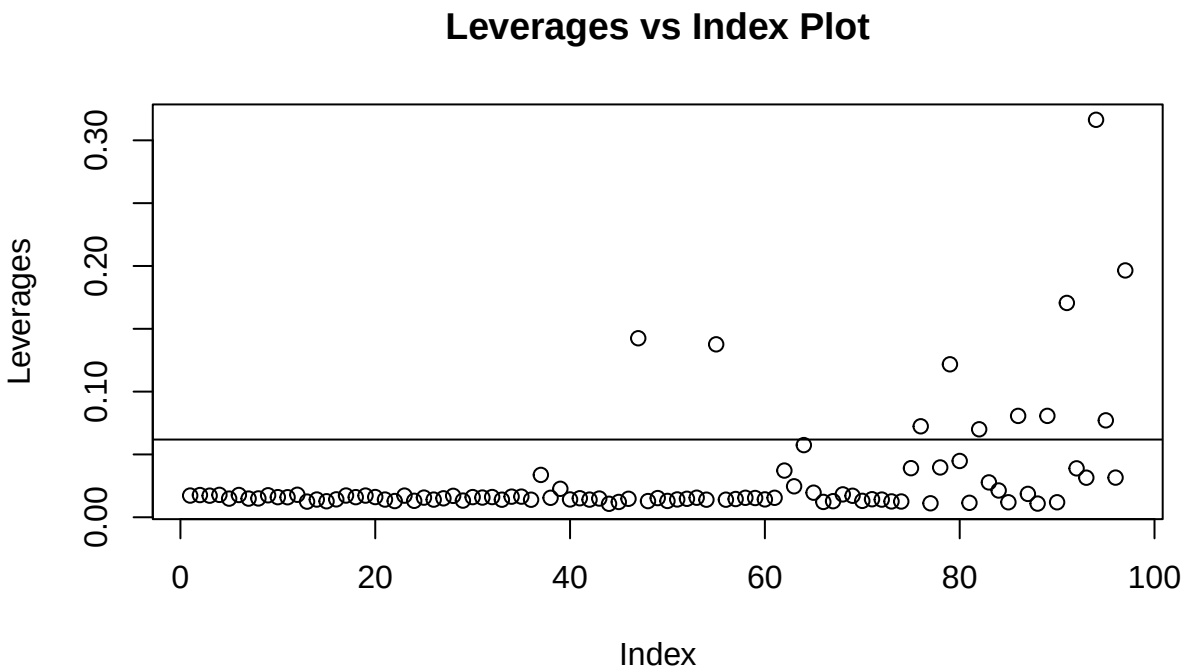

```
cook <- cooks.distance(f1.1)
plot(cook, ylab="Cooks distance")
```



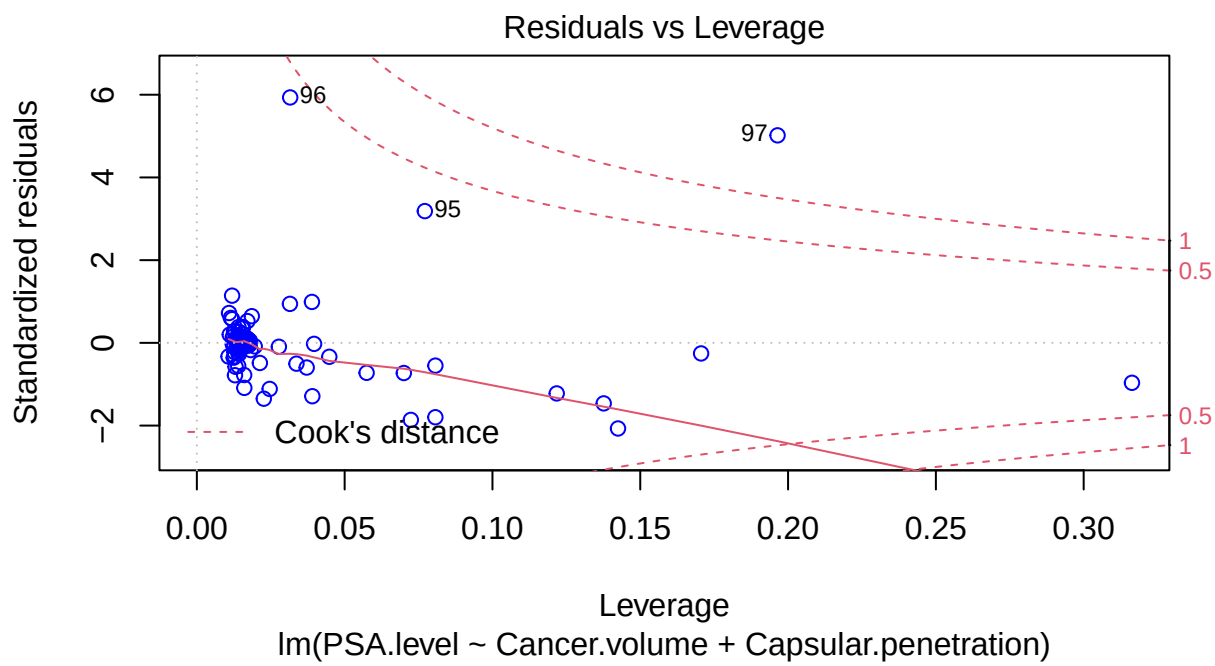

```
ols_plot_dffits(f1.1)
```




```
hatv <- hatvalues(f1.1)
plot(hatv, ylab="Leverages",main="Leverages vs Index Plot")
abline(h=2*p/n)
```

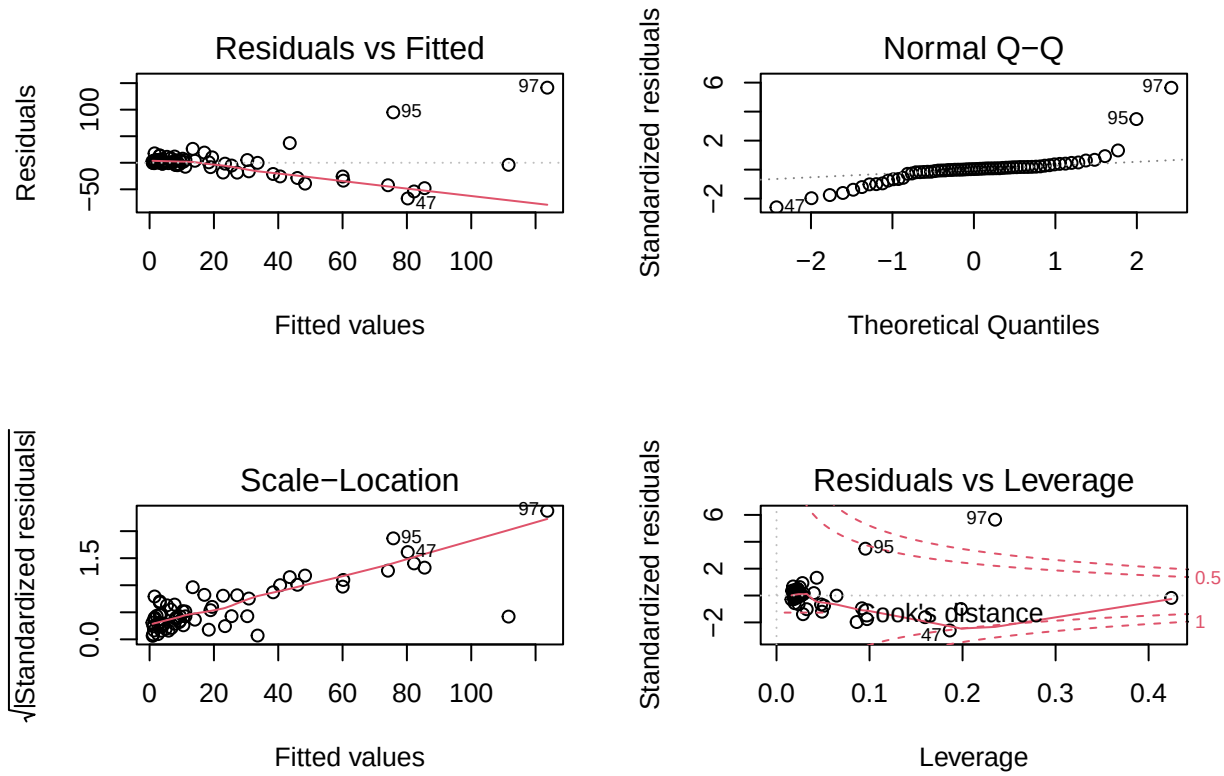



```
plot(f1.1, which=5, col=c("blue"))
```



The points 95,96,97 are identified as outliers or at least as very influential points. But given a small dataset, it is hard to justify the removal of these points, so we will keep them in our dataset and continue.

```
par(mfrow=c(2,2))
plot(m2)
```




```
vif(m2)
```

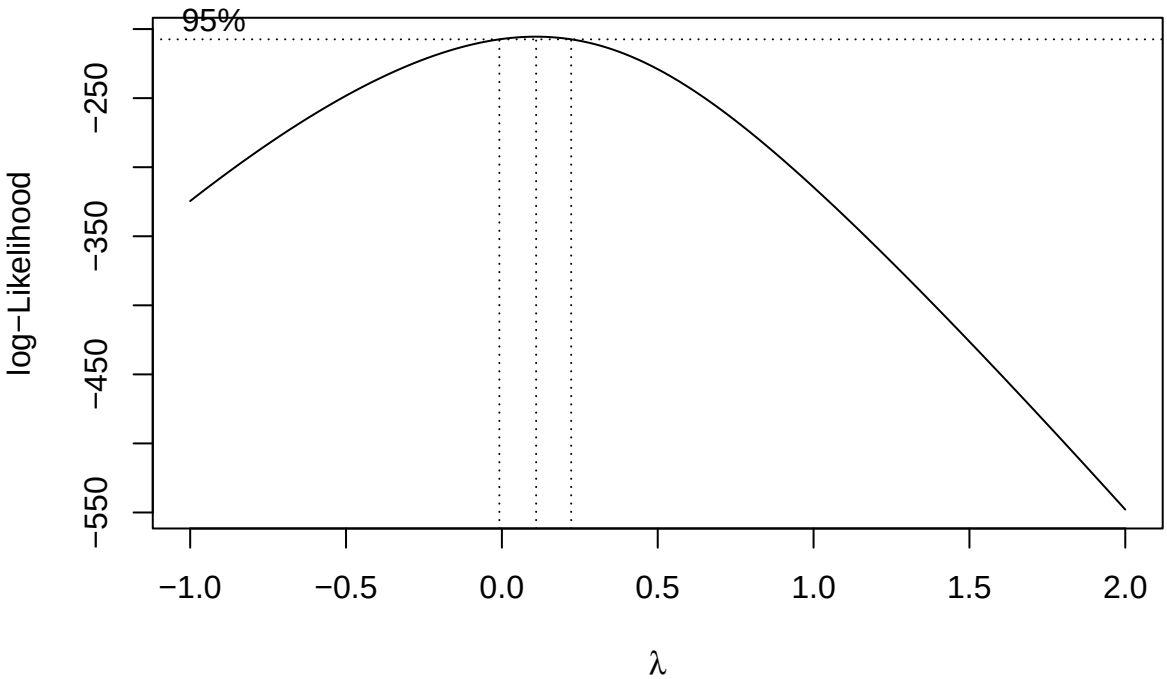
```
##        Cancer.volume Capsular.penetration
##             2.190807             2.190807
```

```
shapiro.test(m2$residuals)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  m2$residuals
## W = 0.74153, p-value = 0.000000002291
```

The residual vs fitted plot and the scale-location plot indicate nonconstant error variances. The QQ plot and Shapiro-Wilk test show non-normality. There is no multicollinearity issue. Let us try the transformation and see if we can remediate these.

```
f.full <- lm(PSA.level ~ Cancer.volume+Capsular.penetration, data=prostate)
bc <- boxcox(f.full,seq(-1,2,0.01))
```



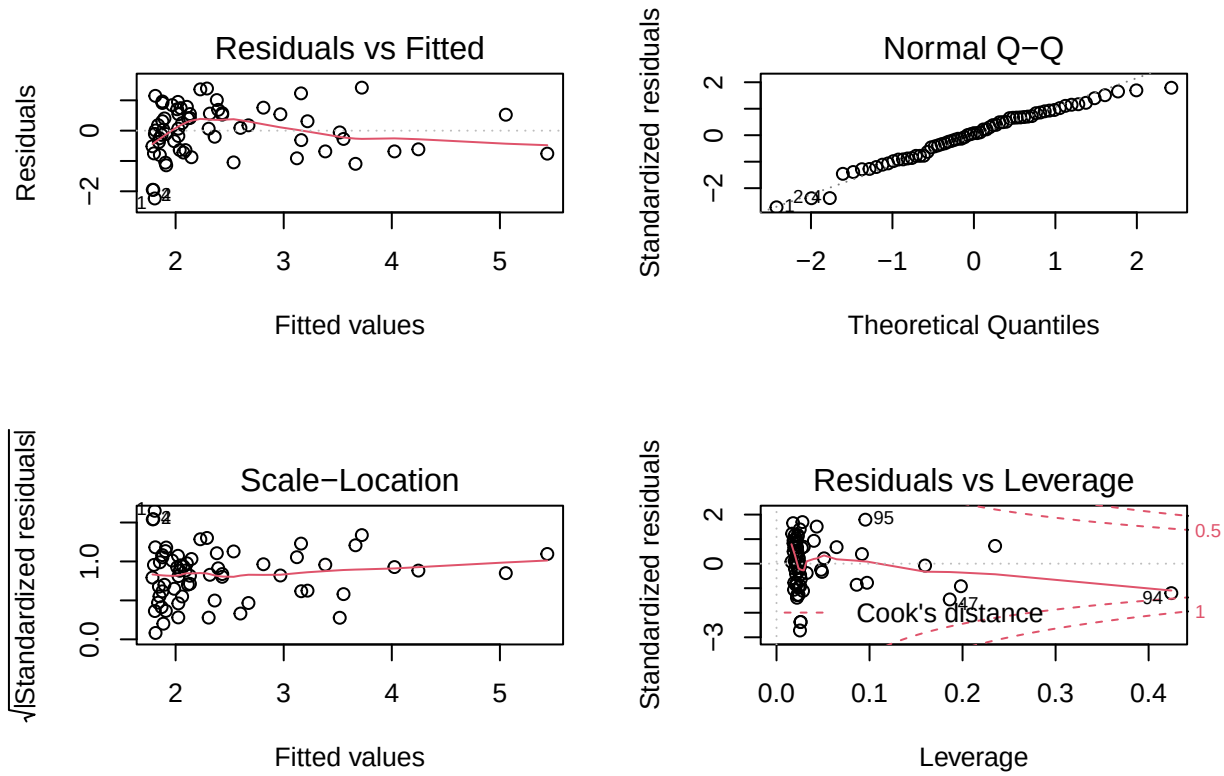

```
best.lambda <-bc$x[which(bc$y==max(bc$y))]
best.lambda
```

```
## [1] 0.11
```

```
f1.log <- lm(log(PSA.level) ~ Cancer.volume+Capsular.penetration, data = train.set)
summary(f1.log)
```

```
##
## Call:
## lm(formula = log(PSA.level) ~ Cancer.volume + Capsular.penetration,
##     data = train.set)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.23615 -0.63842  0.06402  0.56288  1.41745
##
## Coefficients:
##                      Estimate Std. Error t value Pr(>|t|)
## (Intercept)           1.76811    0.13655  12.948  < 2e-16 ***
## Cancer.volume         0.06927    0.01778   3.897 0.000242 ***
## Capsular.penetration  0.05833    0.03677   1.587 0.117705
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8319 on 62 degrees of freedom
## Multiple R-squared:  0.4865, Adjusted R-squared:   0.47
## F-statistic: 29.38 on 2 and 62 DF,  p-value: 0.000000001061
```

```
par(mfrow=c(2,2))
plot(f1.log)
```




```
vif(f1.log)
```

```
##        Cancer.volume Capsular.penetration
##             2.190807             2.190807
```

```
shapiro.test(f1.log$residuals)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  f1.log$residuals
## W = 0.97159, p-value = 0.1399
```

```
p.ols <- resampleMetrics(exp(predict(f1.log, test.set)), test.set$PSA.level)
p.ols
```

```
##               R2     RMSE      MAE
## [1,] 0.05396813 40.92855 17.28827
```

The best λ for Box-Cox procedure is 0.11, but since $Y^{0.11}$ transformation is complicated, we choose the log transformation, which seem to remediate the problems we had. Both constant error variance and normality assumption now hold, and there is no multicollinearity. However, now the performance of the model is poor with R^2 of less than 10%. With this poor performance, it would not be useful for the oncologists to predict PSA levels. We could try Ridge, Lasso, or robust regression to mitigate the influence of outliers, but for the sake of this problem, we settle with the best subset model and move on.

b-)

```
f.tree <- rpart(PSA.level ~ ., data = train.set)
f.tree
```

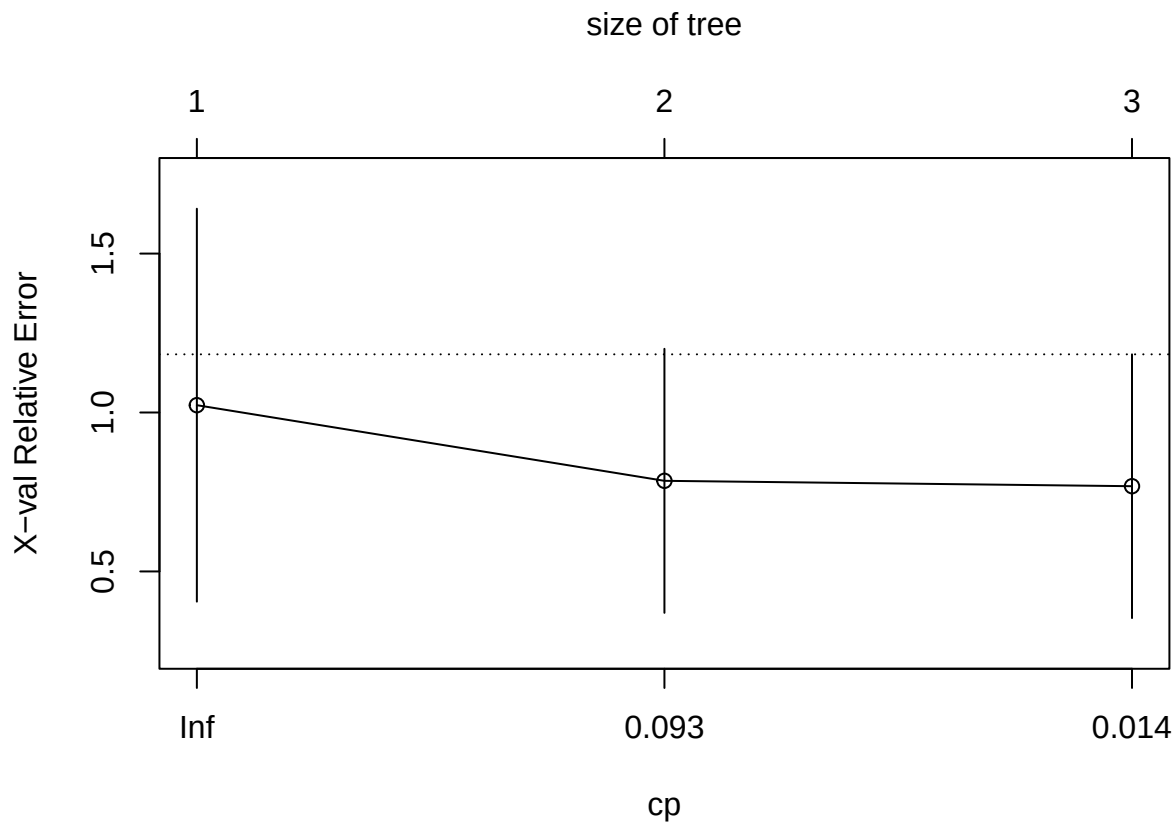
```
## n= 65
##
```

```
## node), split, n, deviance, yval
##       * denotes terminal node
##
## 1) root 65 102729.800 22.539650
##   2) Cancer.volume< 16.53225 57   5642.784 12.868260
##     4) Cancer.volume< 4.7117 36   1154.106  8.230694 *
##     5) Cancer.volume>=4.7117 21   2387.133 20.818380 *
##   3) Cancer.volume>=16.53225 8  53768.320 91.448250 *
```

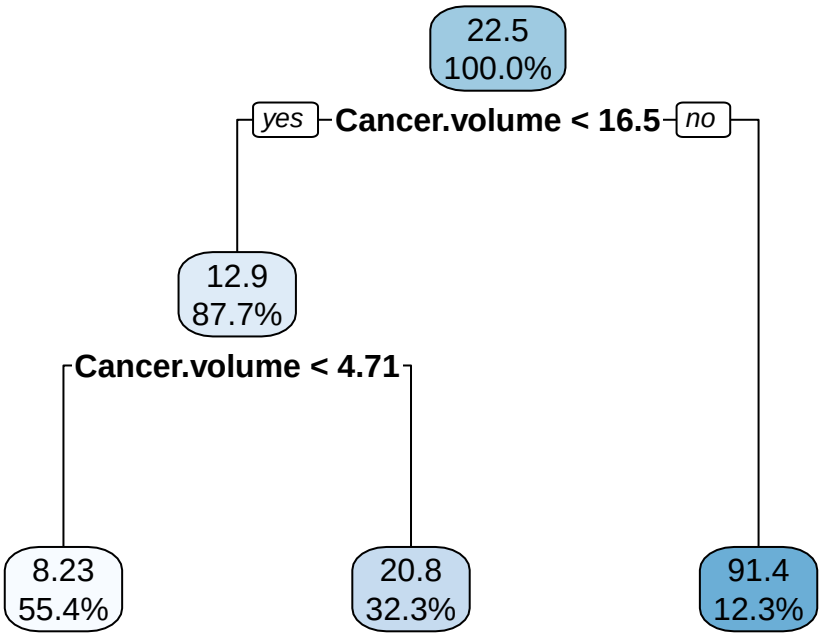
```
f.tree$cptable
```

```
##           CP nsplit rel error    xerror      xstd
## 1 0.42167605      0 1.0000000 1.0230257 0.6180722
## 2 0.02045702      1 0.5783240 0.7850561 0.4153322
## 3 0.01000000      2 0.5578669 0.7681488 0.4148888
```

```
plotcp(f.tree)
```




```
rpart.plot(f.tree, digits=3)
```




```
p.tree <- resampleMetrics(predict(f.tree,test.set), test.set$PSA.level)
```

Although the C_p table suggests that we prune the tree since the entire tree falls into the minimum 'xerror'+ 1 SE, but by this standard, we prune too much and have no tree at all. Therefore, we keep the original tree as is, and it is already a simple tree.

The regression tree only considers *Cancer.volume*, and if it is greater than or equal to 16.53, PSA level is predicted to be 91.45. When it is less than 4.71, PSA level = 8.23, and when greater than or equal to 4.71 and less than 16.53, PSA level = 20.82.

c-)

```
table1 <- rbind(p.ols, p.tree)
rownames(table1) <- c("OLS","Regression Tree")
kable(table1, digits=3, align="c", booktabs=TRUE,caption="Model Comparison") %>%
  kable_styling(latex_options="HOLD_position")
```

Table 1: Model Comparison

	R2	RMSE	MAE
OLS	0.054	40.929	17.288
Regression Tree	0.465	30.787	16.018

The regression tree performs better and is easier to interpret since it only takes into account one variable and also has the plot of the tree. Also, the best regression model is log transformed, which makes it harder to interpret, and interpreting the regression function is more complicated at least for this problem, given such a simple regression tree.

d-) Neural networks work best when the input data are scaled to a narrow range around zero, so we first normalize each column of the dataset. The normalization also leads to faster learning and convergence.

```
normalize <- function(x) {return((x - min(x)) / (max(x) - min(x)))}
prostate_norm <- as.data.frame(lapply(prostate, normalize))
```

```r
train_norm <- prostate_norm[train.index,]
test_norm <- prostate_norm[-train.index,]

softplus <- function(x) { log(1 + exp(x)) }
set.seed(567)
NN <- neuralnet(PSA.level ~ ., data = train_norm, hidden = 4, act.fct = softplus)
plot(NN)
NN_result <- compute(NN, test_norm[2:8])
pred.NN <- NN_result$net.result
cor(pred.NN, test_norm$PSA.level)
```

```
##           [,1]
## [1,] 0.2590617
```

```r
unnormalize <- function(x) {return((x * max(prostate$PSA.level) - min(prostate$PSA.level)) + min(prosta
p.NN <- resampleMetrics(unnormalize(pred.NN), test.set$PSA.level)

# check in-sample performance to see if there's overfitting
NN_result2 <- compute(NN, train_norm[2:8])
pred.NN2 <- NN_result2$net.result
cor(pred.NN2, train_norm$PSA.level)
```

```
##           [,1]
## [1,] 0.9767016
```

```r
p.NN2 <- resampleMetrics(unnormalize(pred.NN2), train.set$PSA.level)
```

ReLU is the most popular activation function, and it is what we also want to use, but since it doesn't work with neuralnet() function, we will instead use a smooth approximation of ReLU, i.e. softplus.

For the choice of number of hidden layers and nodes, we first choose just one hidden layer, given our small dataset. Four hidden nodes are chosen since it is customary to choose between the number of input nodes and that of output nodes, which are 7 and 1 in this case.

The interpretation of a neural network is very difficult, but based on what we see in the plot above, each of the 7 input nodes is sent to each of the hidden nodes with different weights and biases. Each hidden node converts these inputs with the activation function, which is softplus in our case, and again with different weights and bias, they are sent to the single output node, *PSA.level*.

The problem with our model is that it significantly overfits the training data. We are already using a small number of hidden nodes, and using only one hidden node does not improve the overfitting. We probably need a lot more data to effectively use the neural network, so for this problem, there appears not much for us to do.

```r
table1.1 <- rbind(p.NN, p.NN2)
rownames(table1.1) <- c("Test Data","Training Data")
kable(table1.1, digits=3, align="c", booktabs=TRUE,caption="Overfitting") %>%
  kable_styling(latex_options="HOLD_position")
```

Table 2: Overfitting

	R2	RMSE	MAE
Test Data	-0.242	46.898	25.089
Training Data	0.954	8.554	5.977

e-)

```r
table2 <- rbind(p.ols, p.tree, p.NN)
rownames(table2) <- c("OLS","Regression Tree","Neural Network")
kable(table2, digits=3, align="c", booktabs=TRUE,caption="Model Comparison") %>%
  kable_styling(latex_options="HOLD_position")
```

Table 3: Model Comparison

	R2	RMSE	MAE
OLS	0.054	40.929	17.288
Regression Tree	0.465	30.787	16.018
Neural Network	-0.242	46.898	25.089

The regression tree is our best model as shown in the table above.

f-) In this problem, we covered three models, ordinary least squares regression, regression tree, and neural network. Of these three, the neural network is the most difficult one to interpret since by nature, this model uses all variables with many weights and biases involved along with the activation function. Between the OLS and the regression tree, which one is easier depends on how complicated the regression tree happens to be. Once the regression tree includes too many splits it gets harder and harder to interpret while OLS just depends on how many variables it uses. As previously mentioned, in this problem, the regression tree is very simple and is easier to interpret.

Problem 2

a-)

```r
AA <- getDataSet("Advertising Agency.csv")
str(AA)
```

```
## 'data.frame':    20 obs. of  2 variables:
##  $ Y: num  220 204 207 222 211 ...
##  $ X: num  2.52 2.17 2.23 2.52 2.31 ...
```

```r
head(AA)
```

```
##       Y     X
## 1 220.4 2.521
## 2 203.9 2.171
## 3 207.2 2.234
## 4 221.9 2.524
## 5 211.3 2.305
## 6 222.7 2.523
```

```r
f2 <- lm(Y ~ X, data=AA)
summary(f2)
```

```
##
## Call:
## lm(formula = Y ~ X, data = AA)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.55515 -0.23700  0.05229  0.56250  0.80657
##
```

```
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  93.6865     0.8229   113.8   <2e-16 ***
## X            50.8801     0.2634   193.1   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.631 on 18 degrees of freedom
## Multiple R-squared:  0.9995, Adjusted R-squared:  0.9995
## F-statistic: 3.73e+04 on 1 and 18 DF,  p-value: < 2.2e-16
```

```r
et <- f2$residuals
dwtest(Y~X, data=AA)
```

```
##
##  Durbin-Watson test
##
## data:  Y ~ X
## DW = 0.97374, p-value = 0.002891
## alternative hypothesis: true autocorrelation is greater than 0
```

H_o: $\rho = 0$
H_a: $\rho > 0$

The Durbin-Watson test shows that the p-value is less than α=0.01, so we reject the null hypothesis. There seems to exist the autocorrelation.

b-)

```r
et1 <- Lag(et, shift = 1)

d1 <- sum(na.omit(et1*et))
d2 <- sum(na.omit(et1^2))
rho <- d1/d2

Ytnew <- AA$Y - rho * Lag(AA$Y, shift = 1)
Xtnew <- AA$X - rho * Lag(AA$X, shift = 1)

f2.1 <- lm(Ytnew ~ Xtnew)
summary(f2.1)
```

```
##
## Call:
## lm(formula = Ytnew ~ Xtnew)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.95813 -0.29553 -0.02312  0.34451  0.60490
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  63.3840     0.5592   113.4   <2e-16 ***
## Xtnew        50.5470     0.2622   192.8   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4546 on 17 degrees of freedom
```

```
##   (1 observation deleted due to missingness)
## Multiple R-squared:  0.9995, Adjusted R-squared:  0.9995
## F-statistic: 3.715e+04 on 1 and 17 DF,  p-value: < 2.2e-16
```

```
b0p <- f2.1$coefficients[1]
b1p <- f2.1$coefficients[2]
sb0p <- summary(f2.1)$coefficients[1,2]
sb1p <- summary(f2.1)$coefficients[2,2]
dwtest(Ytnew ~ Xtnew)
```

```
##
##  Durbin-Watson test
##
## data:  Ytnew ~ Xtnew
## DW = 1.7612, p-value = 0.2337
## alternative hypothesis: true autocorrelation is greater than 0
```

The Durbin-Watson test result suggest that we fail to reject the null hypothesis, so after the first iteration of the Cochrane-Orcutt procedure, the autocorrelation seems to be disappeared.

c-)

```
b0 <- b0p / (1-rho)
b1 <- b1p
sb0 <- sb0p / (1-rho)
sb1 <- sb1p
```

After the first iteration of the Cochrane-Orcutt:
$\hat{Y}' = 63.384 + 50.547$X'

In terms of the original variables:
$\hat{Y} = 94.873 + 50.547$X

The coefficients for X and X' are the same, and those for the intercepts are different.

$s(b_0) = 0.8369$
$s(b_1) = 0.2622$

d-)

```
MSE <- summary(f2.1)$sigma^2
Xp <- Xtnew
Xbar.p <- mean(Xp[-1])

X20 <- AA$X[20]
X21 <- 3.625
X21p <- X21 - rho * X20

Yhat21 <- b0 + b1 * X21
Y20 <- AA$Y[20]
e20 <- Y20 - (b0 + b1 * X20)
F21 <- Yhat21 + rho*e20

alpha <- 0.01
n <- length(AA$Y)
s.pred <- sqrt(MSE*(1 + (1/n) + (X21p - Xbar.p)^2/(sum((Xp[-1] - Xbar.p)^2))))
s.pred
```

```
## [1] 0.4737689
```

```
pred.L <- F21 - qt(1-alpha/2, df=n-3)*s.pred
pred.U <- F21 + qt(1-alpha/2, df=n-3)*s.pred

print(paste(100*(1-alpha) ,"percent prediction interval for response at month 21 is:", round(pred.L,4),
```

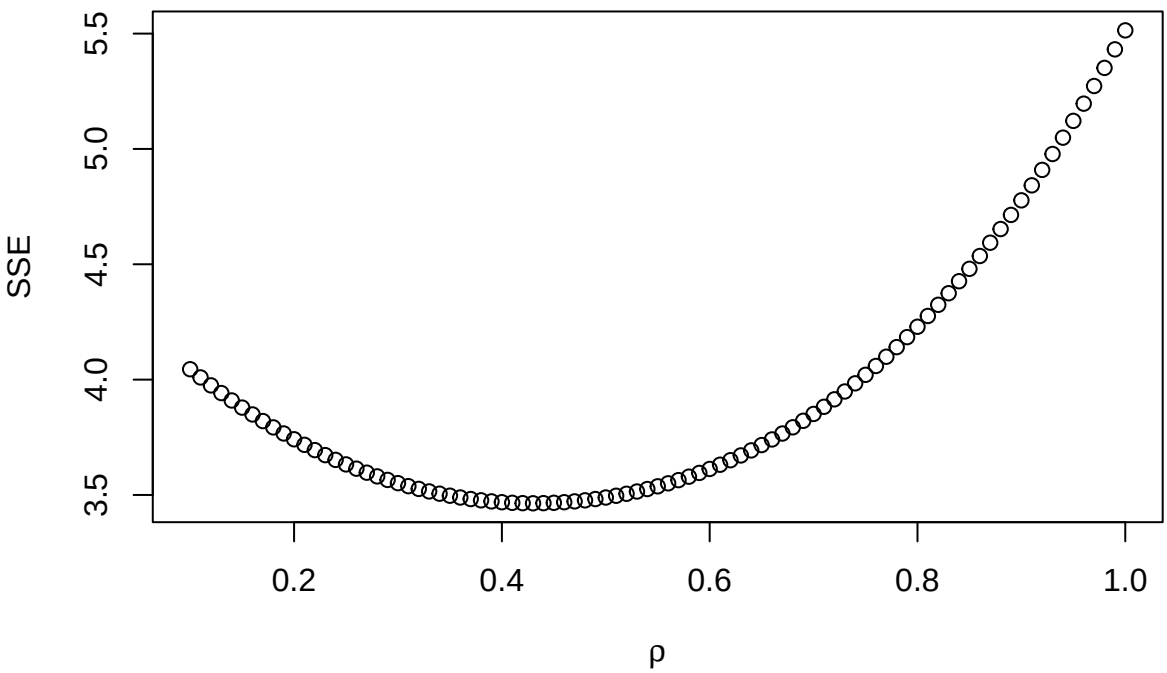
```
## [1] "99 percent prediction interval for response at month 21 is: 276.9807 , 279.7268"
```

Given the staff time in month 21 as 3.625 thousand hours, and given the amount of billings in month 20 as 278.5 thousand constant dollars, we predict with approximately 99 percent confidence that the amount of billings in month 21 will be between 276.98 and 279.73 thousand constant dollars

Problem 3

a-)

```
hildreth <- function(x,y,rho){
  n3 <- length(rho)
  out <- matrix(0, nrow=n3, ncol=2)
  out[,1] <- rho

  for (i in 1:n3){
    d <- anova(hildreth.lu(y=y, x=x, rho=rho[i]))
    out[i,2] <- d$"Sum Sq"[2]
  }
  out
}
rh <- seq(0.1,1,by=0.01)
hl <- hildreth(AA$X, AA$Y, rh)
plot(hl, xlab=expression(rho), ylab="SSE")
```




```
rho3a <- hl[which.min(hl[,2]),][1]

Ytp <- AA$Y - rho3a * Lag(AA$Y, shift = 1)
```

```r
Xtp <- AA$X - rho3a * Lag(AA$X, shift = 1)

f3.1 <- lm(Ytp ~ Xtp)
summary(f3.1)
```

```
##
## Call:
## lm(formula = Ytp ~ Xtp)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.93572 -0.32605  0.02965  0.37221  0.62660
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  54.2435     0.5207   104.2   <2e-16 ***
## Xtp          50.4659     0.2842   177.6   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4514 on 17 degrees of freedom
##   (1 observation deleted due to missingness)
## Multiple R-squared:  0.9995, Adjusted R-squared:  0.9994
## F-statistic: 3.154e+04 on 1 and 17 DF,  p-value: < 2.2e-16
```

```r
b1.3a <- f3.1$coefficients[2]
sb1.3a <- summary(f3.1)$coefficients[2,2]
```

The value of ρ that minimized SSE is 0.43. The estimated transformed regression function:
$\hat{Y}' = 54.2435 + 50.4659$X'

b-)

```r
rho3b <- 1
Ytp2 <- AA$Y - rho3b*Lag(AA$Y, shift = 1)
Xtp2 <- AA$X - rho3b*Lag(AA$X, shift = 1)
f3.2 <- lm(Ytp2 ~ Xtp2 - 1)
summary(f3.2)
```

```
##
## Call:
## lm(formula = Ytp2 ~ Xtp2 - 1)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -0.8016 -0.1744  0.1508  0.4578  1.0575
##
## Coefficients:
##      Estimate Std. Error t value Pr(>|t|)
## Xtp2   50.164      0.425     118   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5787 on 18 degrees of freedom
##   (1 observation deleted due to missingness)
## Multiple R-squared:  0.9987, Adjusted R-squared:  0.9986
```

```
## F-statistic: 1.393e+04 on 1 and 18 DF,  p-value: < 2.2e-16
```

```
b1.3b <- f3.2$coefficients[1]
b0.3b <- mean(AA$Y) - mean(AA$X)*b1.3b
sb1.3b <- summary(f3.2)$coefficients[1,2]
```

The first difference procedure uses r = 1, and the estimated transformed regression function is:
$\hat{Y}' = 50.164$X'

c-) The Durbin-Watson test results below show that the autocorrelation is successfully remediated for both models.

```
dwtest(f3.1) # Hildreth-Lu
```

```
##
##  Durbin-Watson test
##
## data:  f3.1
## DW = 1.9677, p-value = 0.408
## alternative hypothesis: true autocorrelation is greater than 0
```

```
dwtest(f3.2) # First Difference
```

```
##
##  Durbin-Watson test
##
## data:  f3.2
## DW = 2.182, p-value = 0.7493
## alternative hypothesis: true autocorrelation is greater than 0
```

H_o: $\rho = 0$
H_a: $\rho > 0$

For both cases, the p-value is greater than α=0.01, so we fail to reject the null hypothesis. There seems to be no autocorrelation.

d-)

```
MSE1 <- summary(f3.1)$sigma^2 # Hildreth-Lu
MSE2 <- summary(f3.2)$sigma^2 # First difference

hild <- c(b1.3a, sb1.3a, rho3a, MSE1)
fd <- c(b1.3b, sb1.3b, rho3b, MSE2)
tbl <- data.frame(rbind(hild, fd))
rownames(tbl) <- c("Hildreth-Lu", "First difference")
colnames(tbl) <- c("b1", "s{b1}", "r", "MSE")

kable(tbl, digits=4, align="c",
      booktabs=TRUE) %>% kable_styling(latex_options="HOLD_position")
```

	b1	s{b1}	r	MSE
Hildreth-Lu	50.4659	0.2842	0.43	0.2038
First difference	50.1641	0.4250	1.00	0.3349

Based on the table above, we would choose the Hildreth-Lu since it resulted in smaller MSE and sb_1.

e-)

```r
Xp <- Xtp
Xbar.p <- mean(Xp[-1])

X20 <- AA$X[20]
X21 <- 3.625
X21p <- X21 - rho3a * X20

Yhat21 <- b0 + b1 * X21
Y20 <- AA$Y[20]
e20 <- Y20 - (b0 + b1 * X20)
F21 <- Yhat21 + rho3a * e20

alpha <- 0.01
n <- length(AA$Y)
s.pred <- sqrt(MSE1*(1 + (1/n) + (X21p - Xbar.p)^2/(sum((Xp[-1] - Xbar.p)^2))))
s.pred
```

```
## [1] 0.4690292
```

```r
pred.L <- F21 - qt(1-alpha/2, df=n-3)*s.pred
pred.U <- F21 + qt(1-alpha/2, df=n-3)*s.pred

print(paste(100*(1-alpha) ,"percent prediction interval for response at month 21 is:", round(pred.L,4),
```

```
## [1] "99 percent prediction interval for response at month 21 is: 277.0678 , 279.7865"
```

Given the staff time in month 21 as 3.625 thousand hours, and given the amount of billings in month 20 as 278.5 thousand constant dollars, we predict with approximately 99 percent confidence that the amount of billings in month 21 will be between 277.07 and 279.79 thousand constant dollars.

Problem 4

a-) There are many zeros in the dataset that do not actually make sense. We consider these as recording errors and remove them.

```r
data(pima)
str(pima)
```

```
## 'data.frame':    768 obs. of  9 variables:
##  $ pregnant : int  6 1 8 1 0 5 3 10 2 8 ...
##  $ glucose  : int  148 85 183 89 137 116 78 115 197 125 ...
##  $ diastolic: int  72 66 64 66 40 74 50 0 70 96 ...
##  $ triceps  : int  35 29 0 23 35 0 32 0 45 0 ...
##  $ insulin  : int  0 0 0 94 168 0 88 0 543 0 ...
##  $ bmi      : num  33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 0 ...
##  $ diabetes : num  0.627 0.351 0.672 0.167 2.288 ...
##  $ age      : int  50 31 32 21 33 30 26 29 53 54 ...
##  $ test     : int  1 0 1 0 1 0 1 0 1 1 ...
```

```r
head(pima)
```

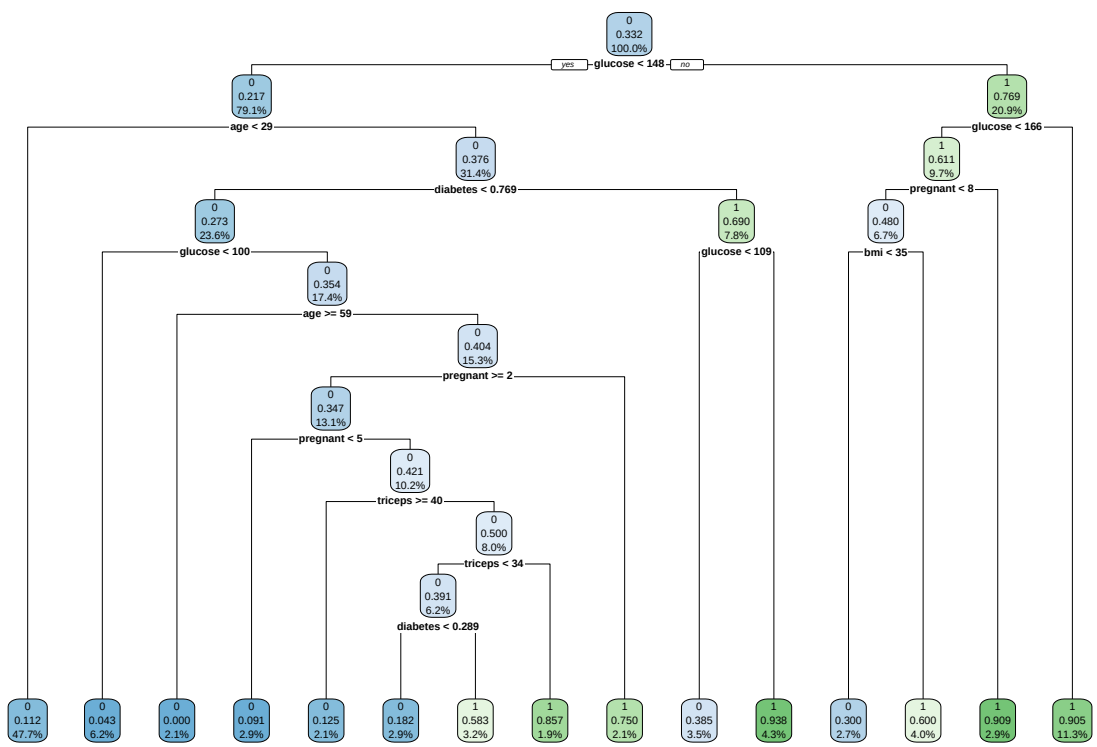
```
##   pregnant glucose diastolic triceps insulin  bmi diabetes age test
## 1        6     148        72      35       0 33.6    0.627  50    1
## 2        1      85        66      29       0 26.6    0.351  31    0
## 3        8     183        64       0       0 23.3    0.672  32    1
## 4        1      89        66      23      94 28.1    0.167  21    0
## 5        0     137        40      35     168 43.1    2.288  33    1
```

```
## 6         5     116         74        0        0 25.6     0.201  30     0
```

```r
# Set 0's to NA since 0 values do not make sense
pima$diastolic[pima$diastolic == 0] <- NA
pima$glucose[pima$glucose == 0] <- NA
pima$triceps[pima$triceps == 0] <- NA
pima$insulin[pima$insulin == 0] <- NA
pima$bmi[pima$bmi == 0] <- NA
pima$test <- as.factor(pima$test)
apply(pima,2,function(x) sum(is.na(x)))
```

```
##  pregnant   glucose diastolic   triceps   insulin       bmi  diabetes       age
##         0         5        35       227       374        11         0         0
##      test
##         0
```

```r
pima2 <- na.omit(pima[,-5])

set.seed(456)
train.index4 <- createDataPartition(y = pima2$test, p = 0.7, list = FALSE)
train.set4 <- pima2[train.index4,]
test.set4 <- pima2[-train.index4,]

f.tree4 <- rpart(test ~ ., data=train.set4, method="class")
rpart.plot(f.tree4, digits=3)
```

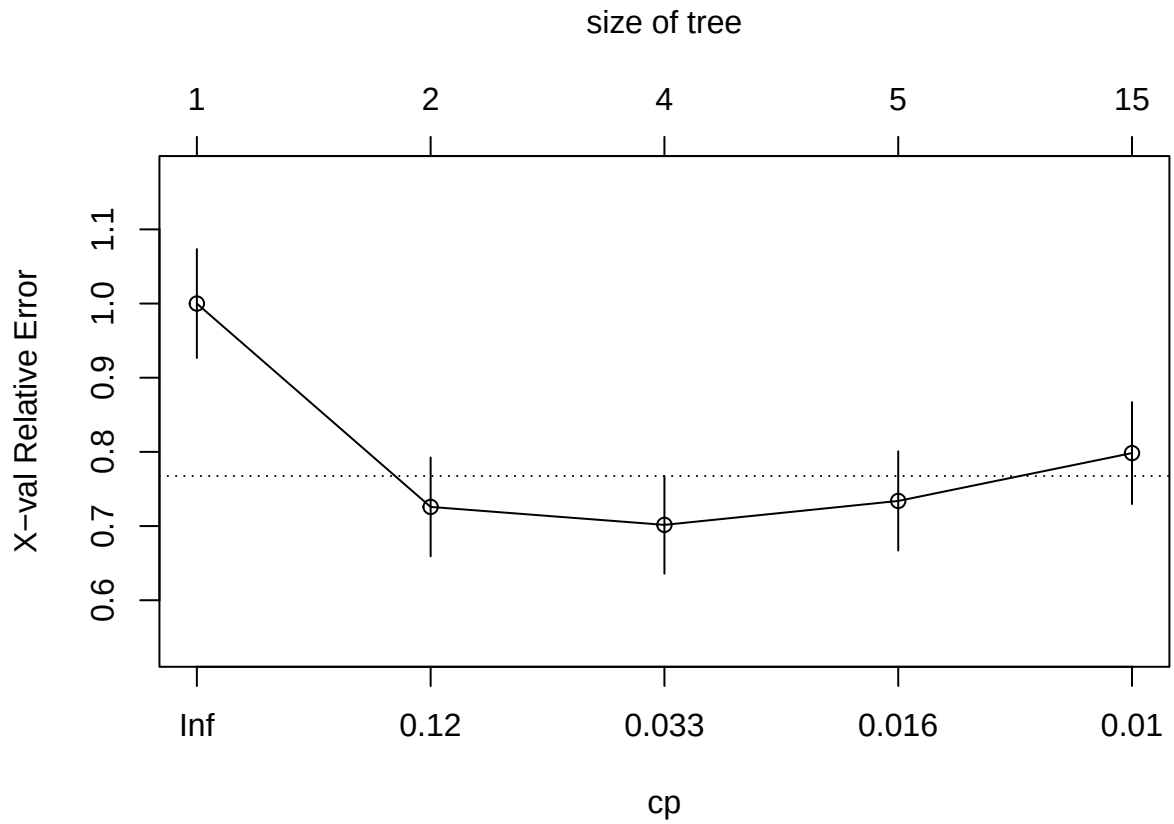



```r
f.tree4$cptable
```

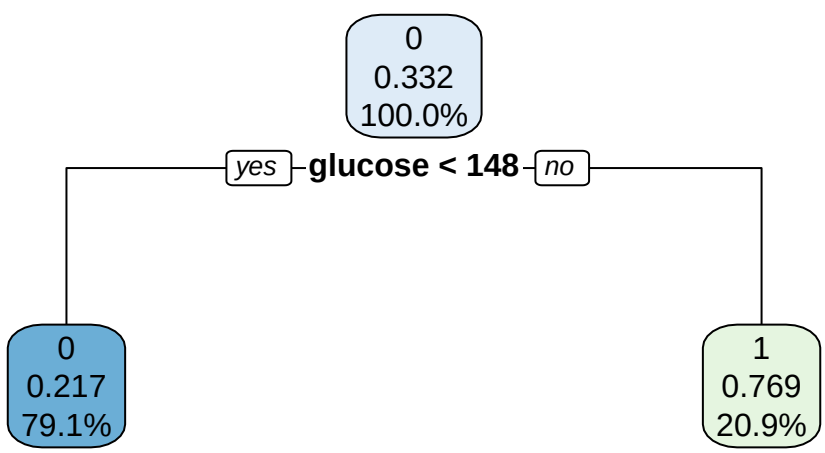
```
##           CP nsplit rel error    xerror       xstd
## 1 0.33870968      0 1.0000000 1.0000000 0.07337269
## 2 0.04435484      1 0.6612903 0.7258065 0.06664052
## 3 0.02419355      3 0.5725806 0.7016129 0.06586680
```

```
## 4 0.01075269      4 0.5483871 0.7338710 0.06689122
## 5 0.01000000     14 0.4274194 0.7983871 0.06877281
```

```
plotcp(f.tree4)
```



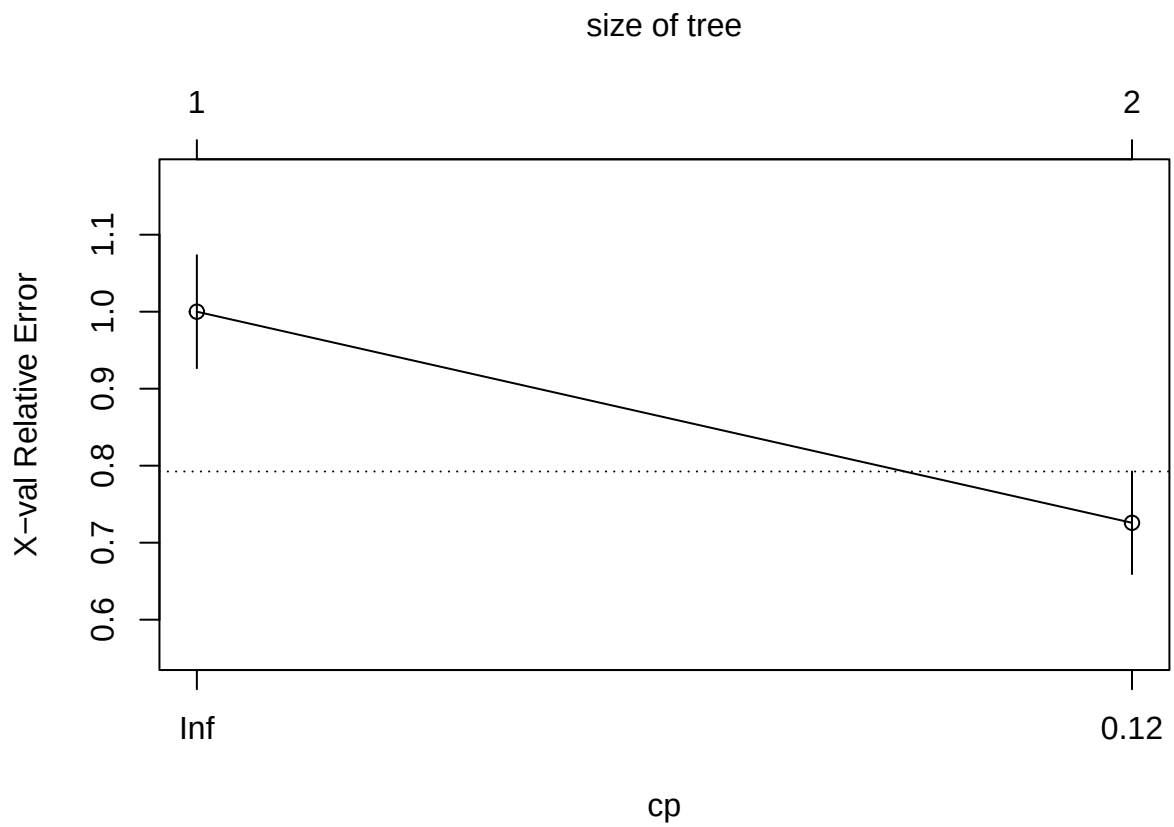

```
a4.1 <- defaultSummary(data.frame(obs = pima2$test, pred = predict(f.tree4, pima2, type="class")))

f.pruned4 <- prune(f.tree4, cp = getBestCp(f.tree4))
rpart.plot(f.pruned4, digits=3)
```




```
f.pruned4$cptable
```

```
##           CP nsplit rel error    xerror       xstd
## 1 0.33870968      0 1.0000000 1.0000000 0.07337269
## 2 0.04435484      1 0.6612903 0.7258065 0.06664052
```

```
plotcp(f.pruned4)
```




```
a4.2 <- defaultSummary(data.frame(obs = pima2$test, pred = predict(f.pruned4, pima2, type="class")))
table4 <- rbind(a4.1, a4.2)
rownames(table4) <- c("Regression Tree","Pruned Regression Tree")
kable(table4, digits=3, align="c", booktabs=TRUE,caption="Model Comparison") %>%
  kable_styling(latex_options="HOLD_position")
```

Table 4: Model Comparison

	Accuracy	Kappa
Regression Tree	0.823	0.587
Pruned Regression Tree	0.769	0.423

The C_p table for the regression tree suggests that we prune the tree, but the pruned tree happens to be overly simple and has only one split. The model comparison table shows that the normal regression tree actually performs better with higher accuary and kaap when evaluated on the full data.

b-)

```
set.seed(456)
bg <- ipred::bagging(test ~ ., data = train.set4, nbagg = 25)
pred.bg <- predict(bg, pima2)
b4.1 <- defaultSummary(data.frame(obs = pima2$test, pred = pred.bg))

ctrl <- trainControl(method = "cv", number = 10)
set.seed(456)
```

```r
bgt <- train(form = test ~ .,
                          data = train.set4,
                          method = "treebag",
                          preProcess = c("center", "scale"),
                          metric="Accuracy",
                          tuneLength=20,
                          trControl=ctrl)
pred.bgt <- predict(bgt, pima2)
b4.2 <- defaultSummary(data.frame(obs = pima2$test, pred = pred.bgt))

table5 <- rbind(a4.1, a4.2, b4.1, b4.2)
rownames(table5) <- c("Regression Tree","Pruned Regression Tree","Bagging 1","Bagging 2")
kable(table5, digits=3, align="c", booktabs=TRUE,caption="Model Comparison") %>%
  kable_styling(latex_options="HOLD_position")
```

Table 5: Model Comparison

	Accuracy	Kappa
Regression Tree	0.823	0.587
Pruned Regression Tree	0.769	0.423
Bagging 1	0.934	0.849
Bagging 2	0.923	0.822

We took two different approaches for bagging, both of which performed well with similar accuracy and kappa. The bagging method looks better than the regression tree in predicting the diabetes test results.

c-)

```r
set.seed(456)
ctrl <- trainControl(method = "cv", number = 10)
bst <- train(form = test ~ .,
            data = train.set4,
            method = "blackboost",
            preProcess = c("center", "scale"),
            metric="Accuracy",
            trControl=ctrl)
pred.bst <- predict(bst, pima2)
c4 <- defaultSummary(data.frame(obs = pima2$test, pred = pred.bst))

table6 <- rbind(a4.1, a4.2, b4.1, b4.2, c4)
rownames(table6) <- c("Regression Tree","Pruned Regression Tree","Bagging 1","Bagging 2","Boosting")
kable(table6, digits=3, align="c", booktabs=TRUE,caption="Model Comparison") %>%
  kable_styling(latex_options="HOLD_position")
```

Table 6: Model Comparison

	Accuracy	Kappa
Regression Tree	0.823	0.587
Pruned Regression Tree	0.769	0.423
Bagging 1	0.934	0.849
Bagging 2	0.923	0.822
Boosting	0.795	0.485

The boosting method performed worse than the bagging, so we still keep the bagging model as the best one at this point.

d-)

```
rf <- randomForest(test ~ ., data = train.set4)
pred.rf <- predict(rf, pima2)
d4 <- defaultSummary(data.frame(obs = pima2$test, pred = pred.rf))

table7 <- rbind(a4.1, a4.2, b4.1, b4.2, c4, d4)
rownames(table7) <- c("Regression Tree","Pruned Regression Tree","Bagging 1","Bagging 2","Boosting","Ra
kable(table7, digits=3, align="c", booktabs=TRUE,caption="Model Comparison") %>%
  kable_styling(latex_options="HOLD_position")
```

Table 7: Model Comparison

	Accuracy	Kappa
Regression Tree	0.823	0.587
Pruned Regression Tree	0.769	0.423
Bagging 1	0.934	0.849
Bagging 2	0.923	0.822
Boosting	0.795	0.485
Random Forest	0.927	0.830

The final table shows that the random forest model is also very good at predicting, and the accuracy and kappa are very close to the two bagging models. Strictly based on the numbers shown in the table, the fist bagging model shows the best results, but because the difference is very marginal, we will consider both the bagging and random forest as the final candidate and do further analysis.